

PostgreSQL 16 — Streaming Replication

Parameter Tuning, Restore Recovery & Cleanup

Complete Q&A Reference — Chat Transcript

Item	Detail
PostgreSQL Version	16
Replication Slot	ssncpri
Data Directory	/apps/pgsql_data/16
Replication Type	Streaming Replication
Date	March 20, 2026

Table of Contents

- Topic 1: Root Cause — WAL Segment Removed Error
- Topic 2: Safe Parameter Changes for Large Restore
- Topic 3: Emergency — Slot Deactivated During Active Restore
- Topic 4: Full Cleanup — Reset to Pre-Restore State
- Topic 5: primary_conninfo vs. primary_slot_name
- Topic 6: pg_basebackup Wipes Standby Data Directory
- Topic 7: Slot Already Exists vs. -C Flag
- Topic 8: Why -S Is Needed Even When Slot Exists
- Topic 9: Copying postgresql.conf from /tmp After pg_basebackup
- Topic 10: ALTER SYSTEM RESET ALL — Reload vs. Restart

Topic 1: Root Cause — WAL Segment Removed Error

Q: After changing WAL/checkpoint parameters for a 400 GB restore and restarting both servers, the standby threw 'Requested WAL segment already been removed' and the replication slot was deactivated. What caused this?

Root Cause Chain

- 1. Parameter changes accelerated WAL generation AND recycling.** Larger checkpoint settings cause checkpoints to complete faster, making PostgreSQL consider older WAL segments as no longer needed sooner.
- 2. The 200+ GB restore generated enormous WAL volume.** Creating FKs on 200+ GB means building indexes, validating constraints, and writing all to WAL.
- 3. The standby fell behind during the restart window.** Standby went down, stopped consuming WAL. Primary restarted and began the massive restore.
- 4. The replication slot went inactive.** The dual restart caused the slot's restart_lsn to become stale, and aggressive checkpointing recycled WAL before the standby reconnected.

Diagnostic Queries

```
SELECT slot_name, active, restart_lsn,  
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS  
       lag  
FROM pg_replication_slots;  
  
SHOW max_slot_wal_keep_size; -- If not -1, that's the culprit
```

Fix: Rebuild standby with pg_basebackup (WAL is already gone).

```
SELECT pg_drop_replication_slot('ssncpri'); -- On PRIMARY  
  
pg_basebackup -h <primary> -D /apps/pgsql_data/16 \  
-U replication_user -Fp -Xs -P -R -S ssncpri -C
```

Topic 2: Safe Parameter Changes for Large Restore

Q: What is the safe way to change parameters for faster restore? Which server to restart first?
How to revert? ALTER SYSTEM vs. postgresql.conf?

Use ALTER SYSTEM for temporary tuning. Your postgresql.conf stays untouched. Rollback is ALTER SYSTEM RESET ALL;

Restore-Optimized Values

Parameter	Default	Restore Value
max_wal_size	1GB	20GB
min_wal_size	80MB	5GB
checkpoint_timeout	5min	30min
checkpoint_completion_target	0.9	0.9 (DO NOT lower)
wal_buffers	-1 (auto)	256MB
work_mem	4MB	256MB
maintenance_work_mem	64MB	2GB
wal_keep_size	0	50GB (CRITICAL)
max_slot_wal_keep_size	-1	-1 (CRITICAL)

SQL — All ALTER SYSTEM Commands

```
ALTER SYSTEM SET max_wal_size = '20GB';
ALTER SYSTEM SET min_wal_size = '5GB';
ALTER SYSTEM SET checkpoint_timeout = '30min';
ALTER SYSTEM SET checkpoint_completion_target = 0.9;
ALTER SYSTEM SET wal_buffers = '256MB';
ALTER SYSTEM SET work_mem = '256MB';
ALTER SYSTEM SET maintenance_work_mem = '2GB';
ALTER SYSTEM SET wal_keep_size = '50GB';
ALTER SYSTEM SET max_slot_wal_keep_size = '-1';
```

Restart Order

Phase	First	Second
Applying changes	STANDBY first	PRIMARY second

Phase	First	Second
Reverting changes	STANDBY first	PRIMARY second

Rollback

SQL — One-Command Rollback

```
ALTER SYSTEM RESET ALL; -- On standby first, then primary
-- Then restart each server
```

Topic 3: Emergency — Slot Deactivated During Active Restore

Q: Replication slot ssncpri deactivated mid-restore. Standby way behind. Restore still running. What to do?

GOLDEN RULE: Never stop the active restore. Replication can be rebuilt — lost restore progress cannot.

Diagnose on PRIMARY

```
SELECT slot_name, active, restart_lsn,  
       pg_size_pretty(pg_wal_lsn_diff(pg_current_wal_lsn(), restart_lsn)) AS  
       slot_lag  
FROM pg_replication_slots WHERE slot_name = 'ssncpri';  
  
SELECT pg_walfile_name(restart_lsn) AS needed_wal  
FROM pg_replication_slots WHERE slot_name = 'ssncpri';  
  
SELECT name FROM pg_ls_waldir()  
WHERE name >= '<needed_wal_file>' ORDER BY name LIMIT 5;
```

Scenario A — WAL Still Exists: Reconnect

```
ALTER SYSTEM SET wal_keep_size = '100GB';  
ALTER SYSTEM SET max_slot_wal_keep_size = '-1';  
SELECT pg_reload_conf();  
-- Then restart standby to reconnect
```

Scenario B — WAL Gone: Rebuild After Restore

```
-- Stop standby, drop slot, wait for restore, then:  
SELECT pg_drop_replication_slot('ssncpri');  
-- After restore: pg_basebackup -R -S ssncpri -C
```

Topic 4: Full Cleanup — Reset to Pre-Restore State

Q: WAL files gone, no time to resync. Want to clean up both servers for next week's restore attempt.

Step	Action	Server
1	Stop the standby	STANDBY
2	Cancel/stop active restore	PRIMARY
3	DROP DATABASE target_db WITH (FORCE);	PRIMARY
4	SELECT pg_drop_replication_slot('ssncpri');	PRIMARY
5	ALTER SYSTEM RESET ALL;	PRIMARY
6	Restart the primary	PRIMARY
7	Verify primary is clean	PRIMARY
8	Wipe standby, pg_basebackup -R -S ssncpri -C	STANDBY
9	Start standby, verify streaming	BOTH
10	Final health check	BOTH

```
# Rebuild standby:
sudo systemctl stop postgresql-16
rm -rf /apps/pgsql_data/16/*
pg_basebackup -h <primary> -p 5432 -U replication_user \
  -D /apps/pgsql_data/16 -Fp -Xs -P -R -S ssncpri -C
chown -R postgres:postgres /apps/pgsql_data/16
sudo systemctl start postgresql-16
```

Topic 5: primary_conninfo vs. primary_slot_name

Q: Is it mandatory to give primary_conninfo? Can I use just primary_slot_name instead?

primary_conninfo is MANDATORY. They do completely different things.

Parameter	What It Does	Mandatory?
primary_conninfo	Connection string — HOW to reach primary	YES
primary_slot_name	Slot name — WHICH slot to use after connecting	No, but recommended
standby.signal	File — tells PG this is a standby	YES (must exist)

primary_conninfo is like a phone number. primary_slot_name is like a reserved seat. You need the phone number to make the call; the seat only matters once you're inside.

Topic 6: pg_basebackup Wipes Standby Data Directory

Q: primary_conninfo is in postgresql.auto.conf. Can I skip -R and just rely on my backed-up postgresql.conf?

No. pg_basebackup wipes the entire standby data directory and copies the primary's files. Your old postgresql.auto.conf (with primary_conninfo) gets overwritten with the primary's copy, which has NO primary_conninfo. Without -R, you also lose standby.signal. The standby starts as a standalone primary.

Always use -R with pg_basebackup.

Topic 7: Slot Already Exists vs. -C Flag

Q: What if I already created the replication slot on the primary before running pg_basebackup?

Drop the -C flag. Keep -S.

Slot Status	Command
Slot does NOT exist	pg_basebackup -R -S ssncpri -C (create + use)
Slot already exists	pg_basebackup -R -S ssncpri (use only, no -C)

Using -C when the slot exists causes: error: replication slot "ssncpri" already exists

```
-- Check before running:
SELECT slot_name FROM pg_replication_slots WHERE slot_name = 'ssncpri';
-- No rows = use -C | Row returned = skip -C
```

Topic 8: Why -S Is Needed Even When Slot Exists

Q: Do I still need -S ssncpri even though the slot already exists on the primary?

Yes. -S does two jobs that don't happen automatically:

Job 1 — During backup: Streams WAL through the slot so no WAL gets recycled mid-backup.

Job 2 — Combined with -R: Writes `primary_slot_name = 'ssncpri'` into `postgresql.auto.conf`.

Without -S, the -R flag creates `standby.signal` and `primary_conninfo` but does NOT set `primary_slot_name`.

Command	During Backup	After Standby Starts
-R -S ssncpri	WAL streamed through slot, no gaps	Standby uses slot automatically
-R (no -S)	WAL streamed without slot, risk of gaps	Standby connects but does NOT use any slot

Without -S, having the slot on the primary is like having a reserved parking spot that nobody ever parks in.

Topic 9: Copying postgresql.conf from /tmp After pg_basebackup

Q: I backed up postgresql.conf (with primary_slot_name=ssncpri) to /tmp. Can I copy it back after pg_basebackup?

Yes, it will work. PostgreSQL reads both files at startup: postgresql.conf provides primary_slot_name, postgresql.auto.conf (from -R) provides primary_conninfo, and standby.signal tells PG it's a standby.

BASH — Steps After pg_basebackup

```
cp /tmp/postgresql.conf /apps/pgsql_data/16/postgresql.conf
ls -la /apps/pgsql_data/16/standby.signal
cat /apps/pgsql_data/16/postgresql.auto.conf
grep primary_slot_name /apps/pgsql_data/16/postgresql.conf
chown postgres:postgres /apps/pgsql_data/16/postgresql.conf
sudo systemctl start postgresql-16
```

Watch for conflicts: If primary_slot_name appears in BOTH files, postgresql.auto.conf always wins.

Topic 10: ALTER SYSTEM RESET ALL — Reload vs. Restart

Q: After ALTER SYSTEM RESET ALL, do I need to restart PostgreSQL? Which server first?

It depends on which parameters you changed.

Parameter	Reload Enough?	Restart Required?
max_wal_size	Yes	No
min_wal_size	Yes	No
checkpoint_timeout	Yes	No
work_mem	Yes	No
maintenance_work_mem	Yes	No
wal_keep_size	Yes	No
max_slot_wal_keep_size	Yes	No
synchronous_commit	Yes	No
wal_buffers	NO	YES — restart required

Since you changed wal_buffers — yes, full restart required. Order: STANDBY first, PRIMARY second.

SQL — Check If Restart Required

```
-- Check if restart is needed for any parameter:
SELECT name, setting, pending_restart
FROM pg_settings
WHERE name IN ('max_wal_size', 'wal_buffers', 'work_mem',
               'maintenance_work_mem', 'wal_keep_size');
-- pending_restart = t means restart needed
```